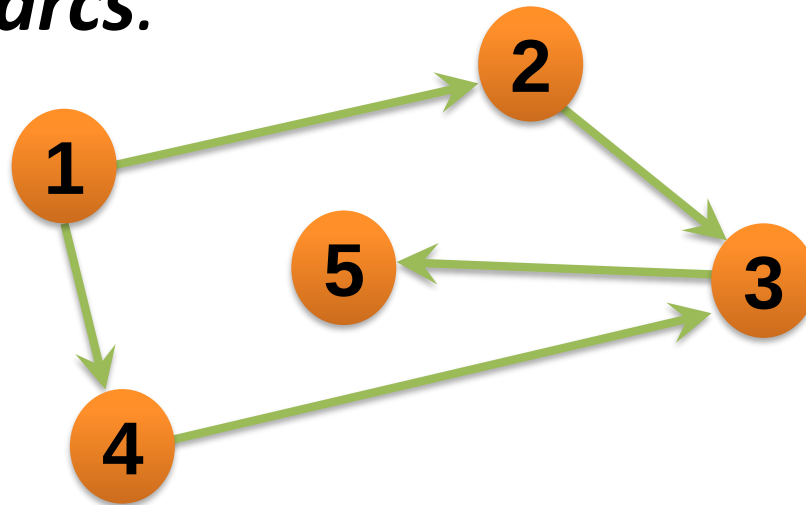


Introduction to Graphs

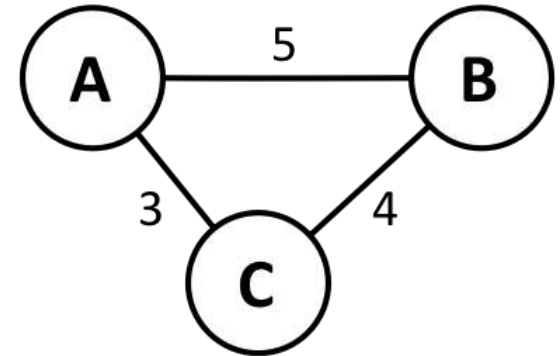
- A graph G is simply a set V of **vertices/nodes** and a collection E of pairs of vertices from V , called **edges/arcs**.



Vertices $v=\{1,2,3,4,5\}$

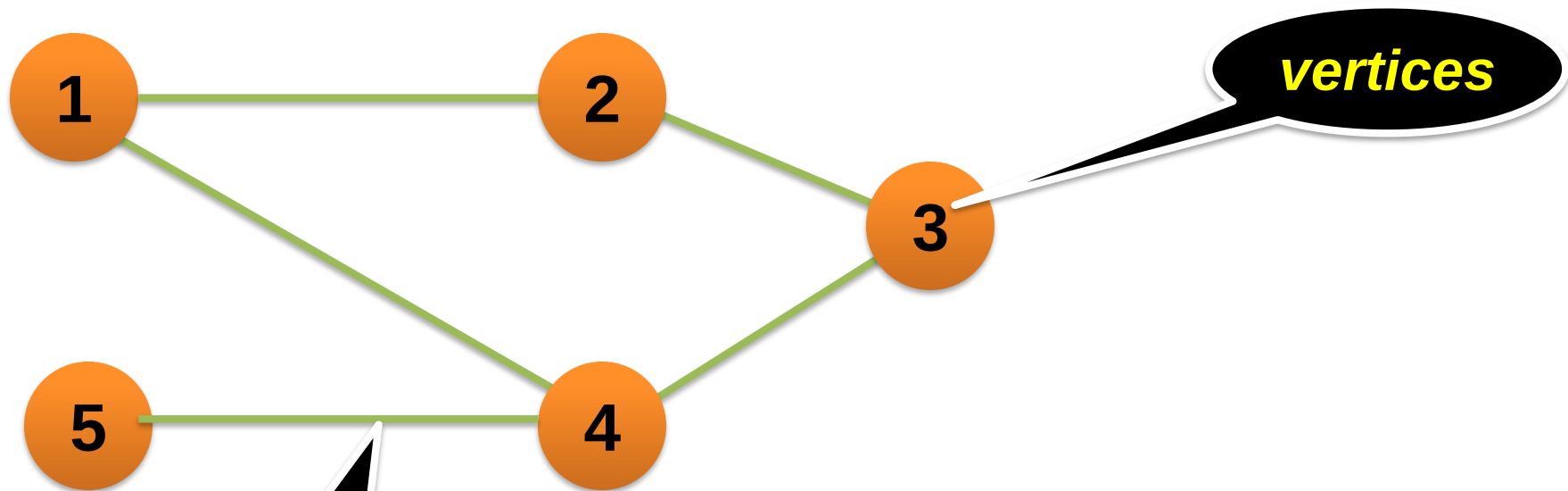
Edges= $\{ (1,4), (1,2), (2,3), (3,5), (4,3) \}$

- Some times edges have third component called weight or cost. Such graphs are called weighted graphs

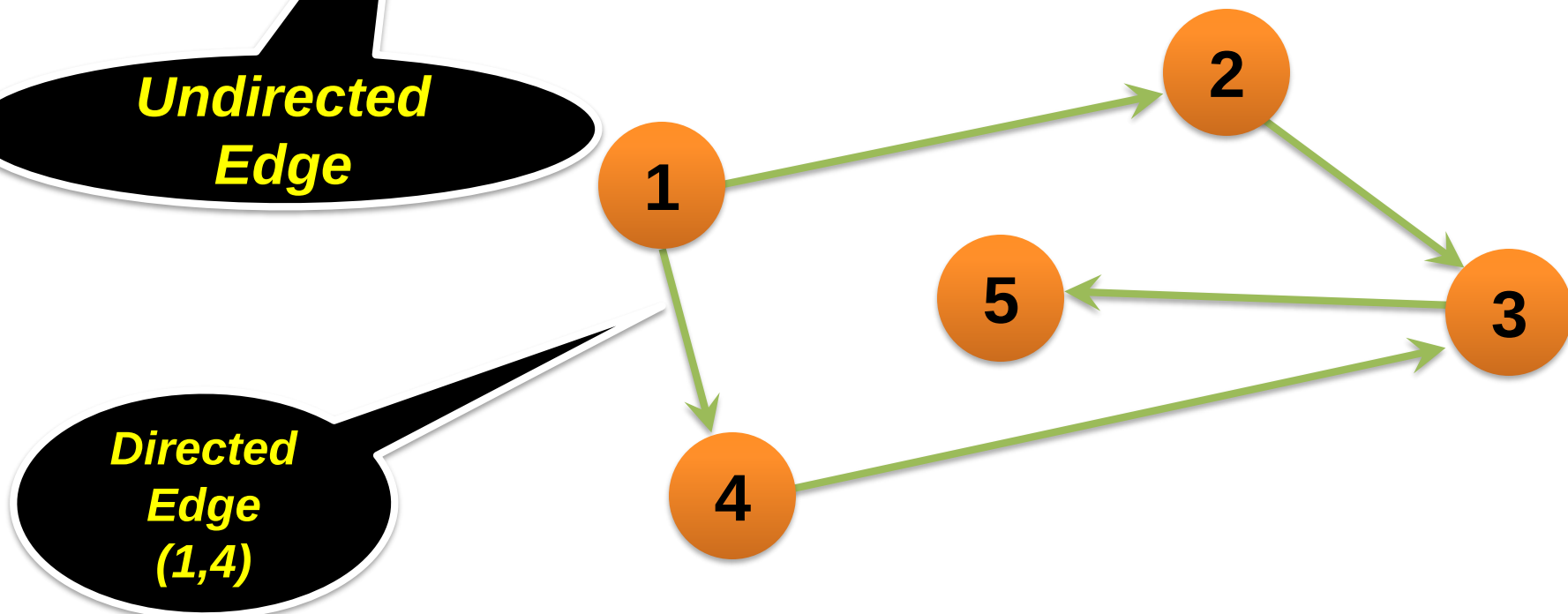


- Edges in a graph are either ***directed or undirected***.
- An edge $\{u,v\}$ is said to be ***undirected*** if
$$\text{edge}(u,v) = \text{edge}(v,u)$$
- An edge (u,v) is said to be ***directed*** if (u,v) and (v,u) edges are not same and represented by two different lines.

- If all the edges in a graph are undirected, then we say the **graph** is an ***undirected graph***.
- a ***directed graph***, also called a ***digraph***, is a graph whose edges are all directed.
- A graph that has both directed and undirected edges is often called a ***mixed graph***

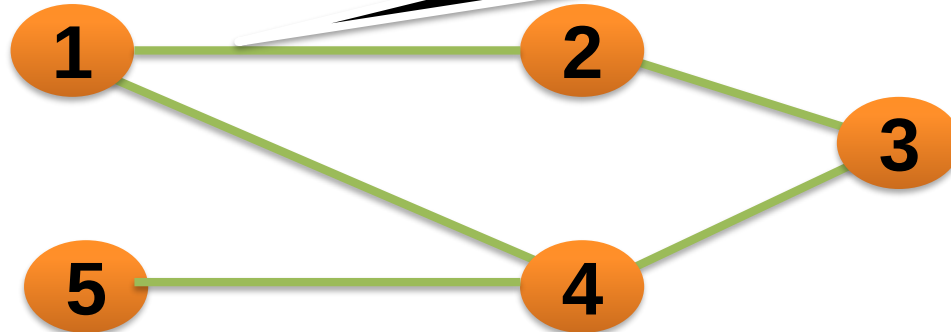


**Undirected
Edge**

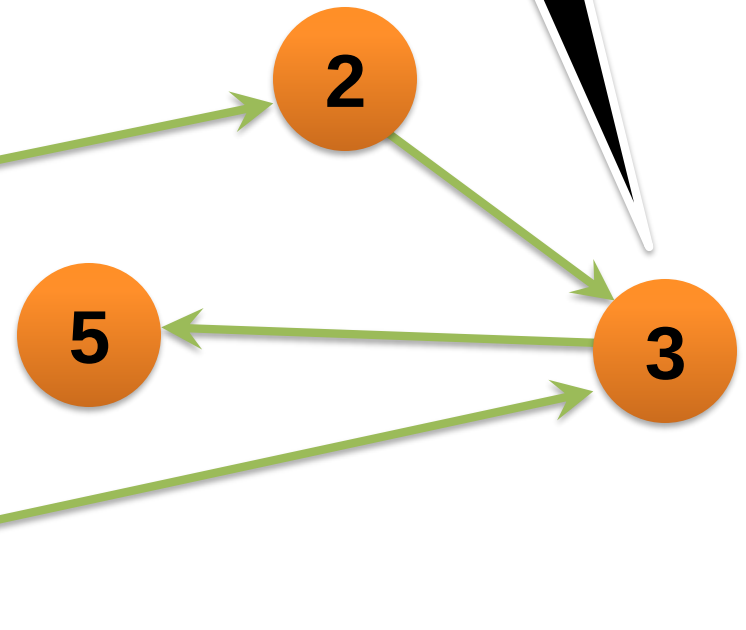


- Vertex u is adjacent to vertex v if there exists an edge between u and v in the graph
- An edge is said to be ***incident*** on a vertex *if* the vertex is one of the edge's endpoints.

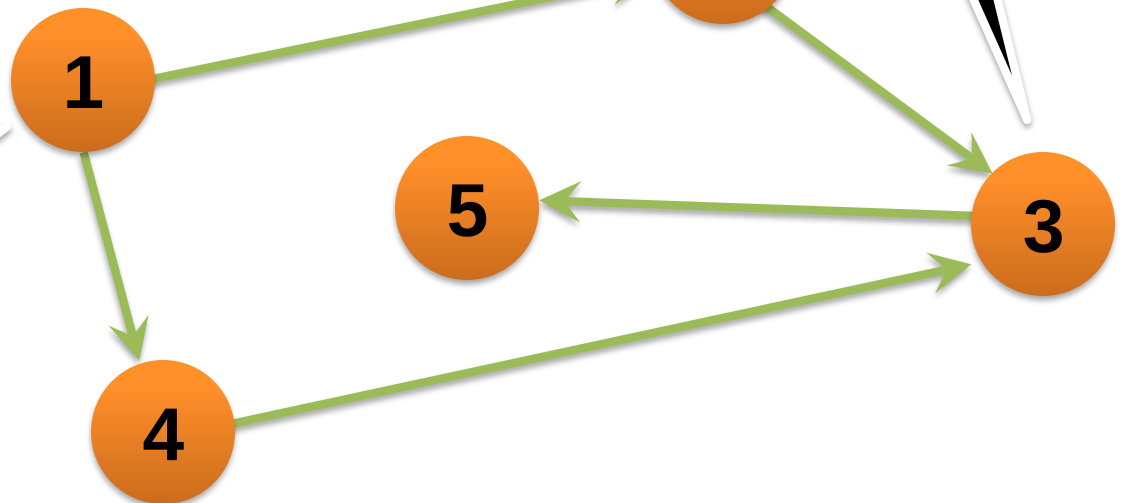
**vertices 1
and 2 are
adjacent**



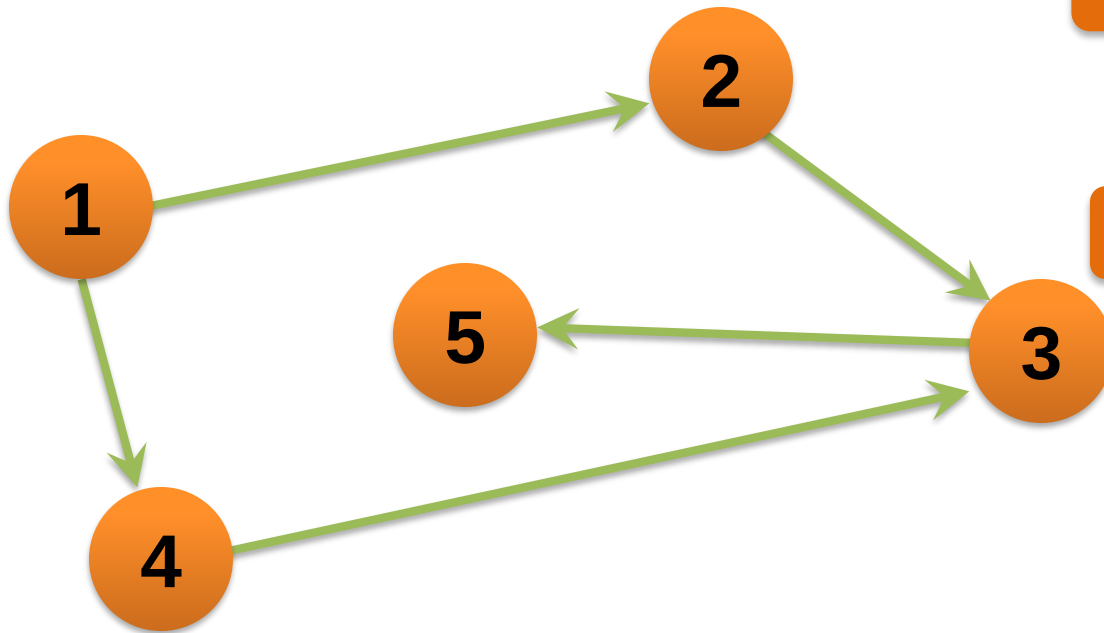
**Incoming
Edges 2**



**Outgoing
Edges 2**



- The **degree** of a vertex v , denoted $\deg(v)$, is the number of incident edges of v . The **in-degree** and **out-degree** of a vertex v are the number of the incoming and outgoing edges of v , and are denoted $\text{indeg}(v)$ and $\text{outdeg}(v)$, respectively.



Degree of 3 is 3

In Degree of 3 is 2

Out Degree of 3 is 1

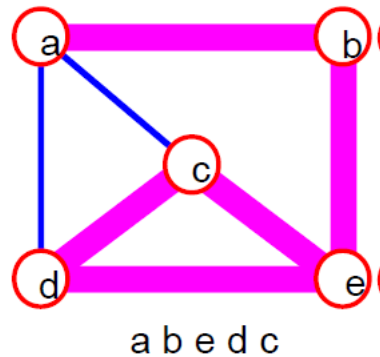
Degree of 4 is 2

In Degree of 4 is 1

Out Degree of 4 is 1

An edge (undirected or directed) is a **self-loop** if its two endpoints coincide

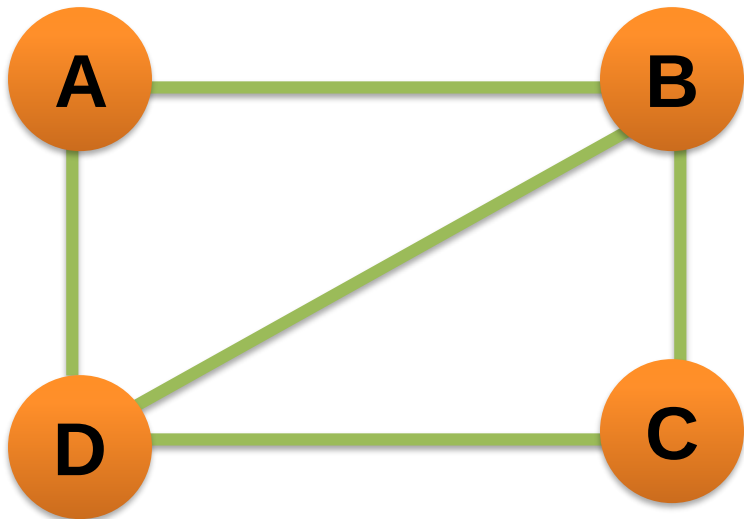
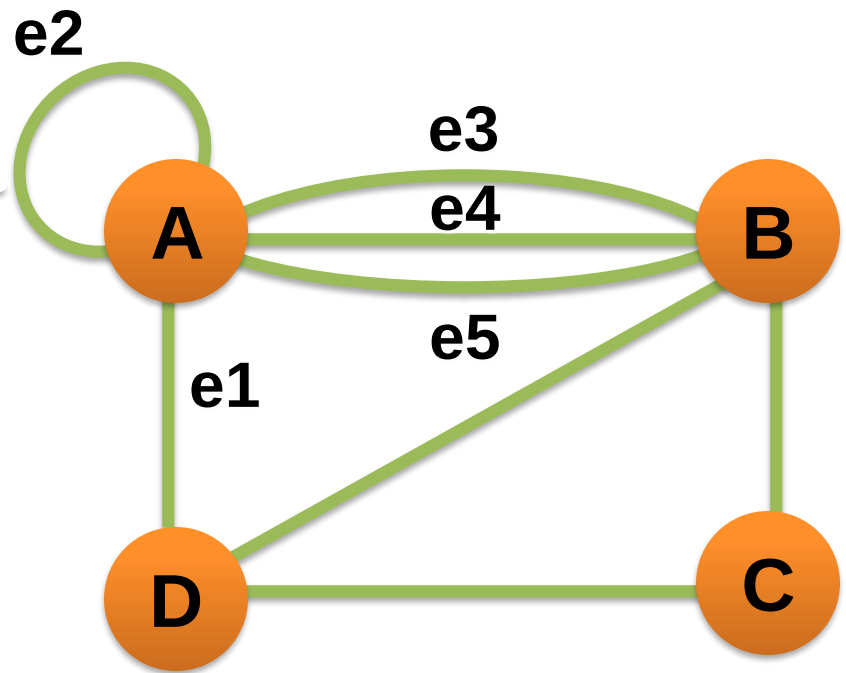
A graphs do not have parallel edges or self-loops such graphs are said to be **simple graph**.



A **path** is a sequence of vertices $v_1, v_2, \dots, v_n$ such that $(v_i, v_{i+1}) \in E$

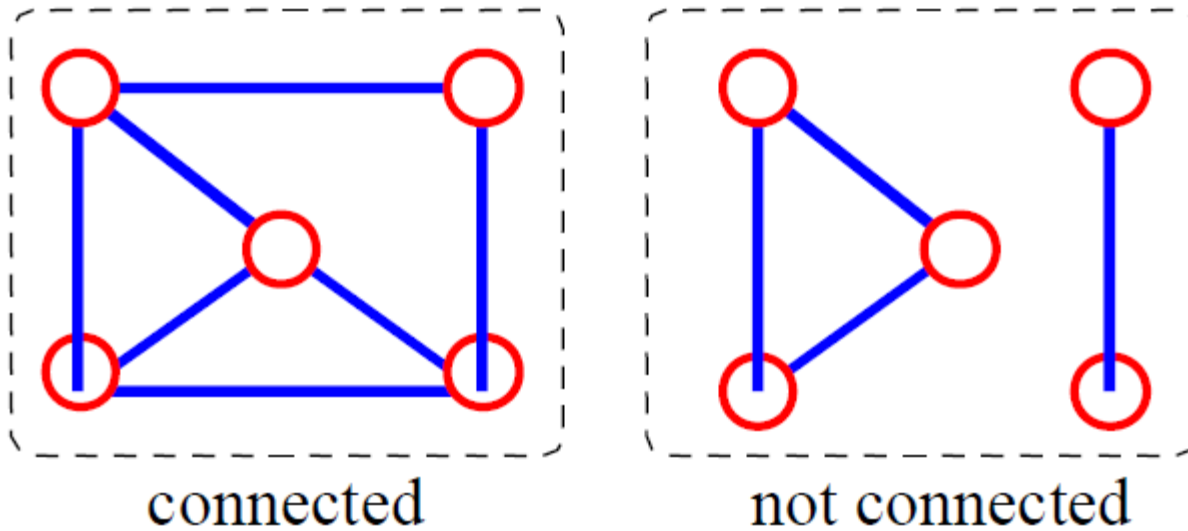
Length of the path is the number of edges on the path

self-loop



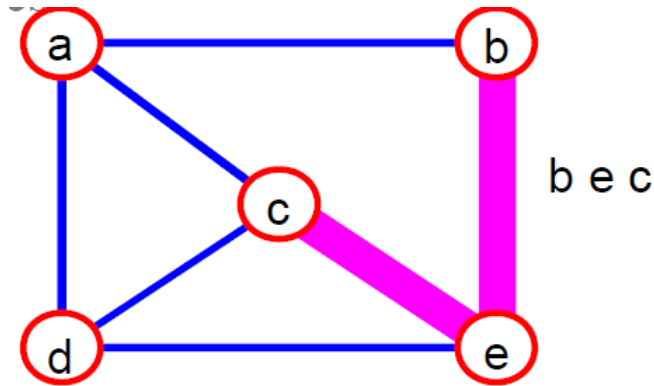
simple

- Connected Graph: any two vertices are connected by some path. All vertices can be reached by all other vertices in graph

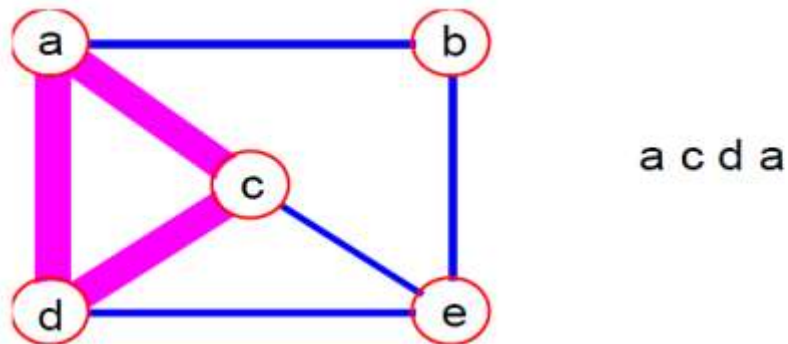


- Tree is a connected graph without cycles

- Simple path : no repeated vertex



Cycle: simple path, except that the last vertex is same as the first



- Directed graphs that have no cycles are called Directed acyclic graphs (DAG)
- A undirected graph is connected if there is a edge from every vertex to every other vertex.
- Directed connected graphs are called strongly connected

Real life situations that can be modelled by a graph

1) Airport System

Vertex : airports

Edges: flight routes

Edge cost: time, distance or cost

Graph is directed graph as cost may not be the same for to and from journey

Make sure the graph is strongly connected so that every airport is connected

We must also determine the best path (having less cost) between any 2 airports

Two vertices are connected by an edge if there exists a non stop flight from the airports

Traffic Flow

- Vertex: each street intersection
- Edge : each street
- Edge cost: speed limit or capacity
- We can find shortest route or most likely location for bottlenecks

Graph Representation

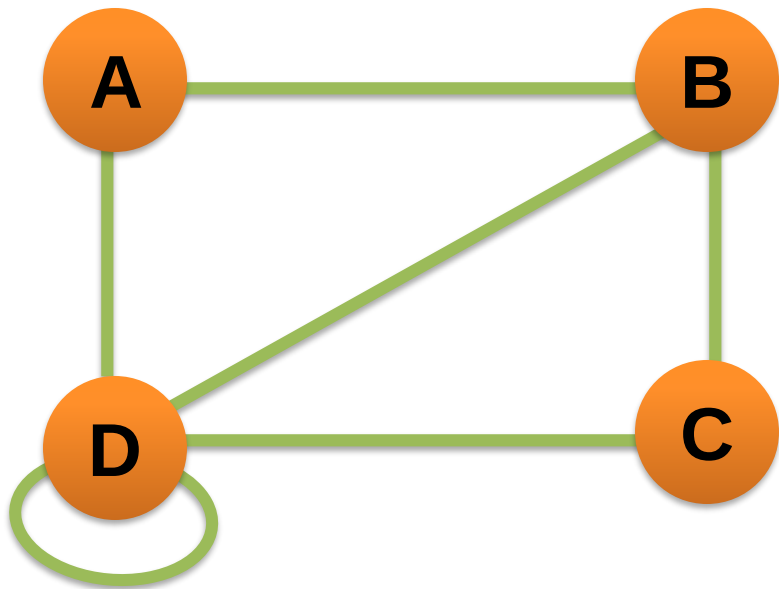
1. Adjacency Matrix

2. Adjacency List

Adjacency Matrix

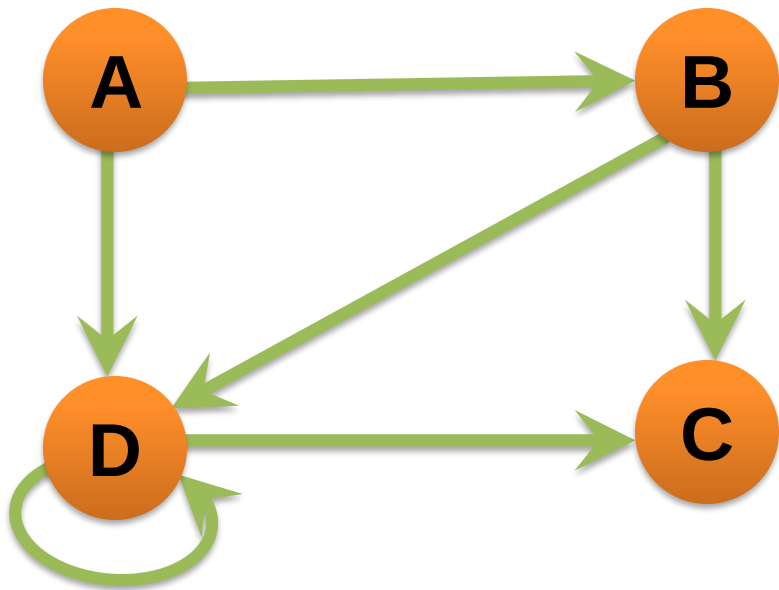
- Assume $V = \{1, 2, \dots, n\}$
- An *adjacency matrix* represents the graph as a $n \times n$ matrix A :
 - $A[i, j] = 1$ if edge $(i, j) \in E$ (or weight of edge)
 $= 0$ if edge $(i, j) \notin E$
 - Storage requirements: $O(V^2)$
 - A dense representation

1. Adjacency Matrix



	A	B	C	D
A	0	1	0	1
B	1	0	1	1
C	0	1	0	1
D	1	1	1	1

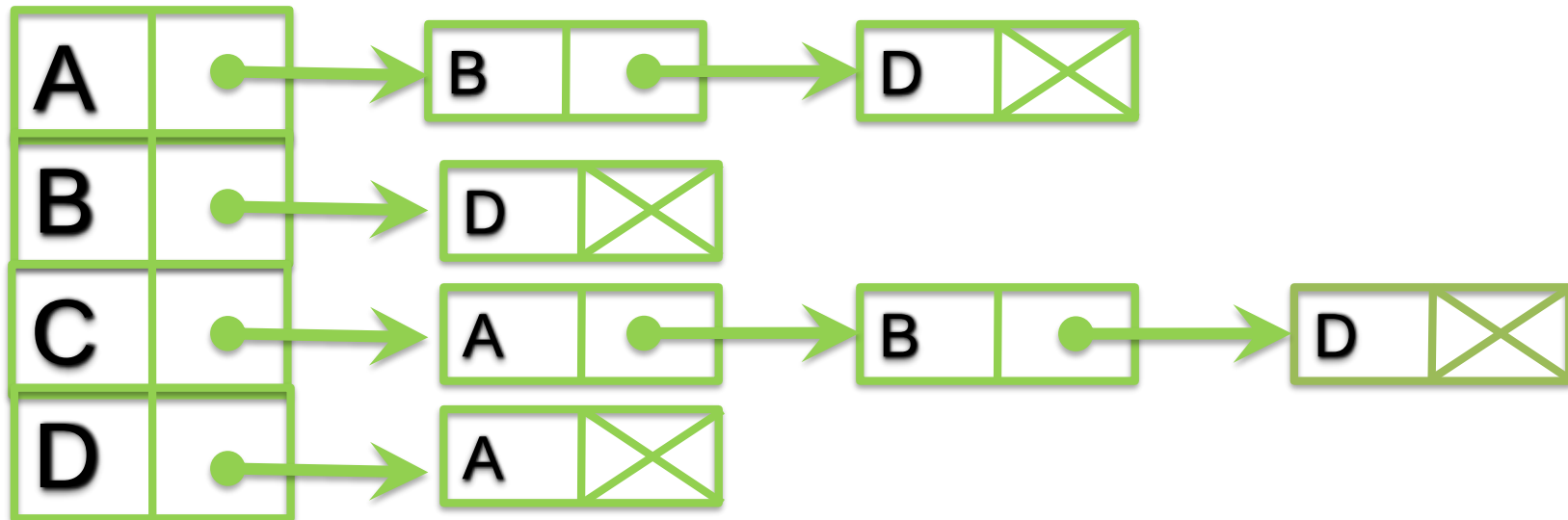
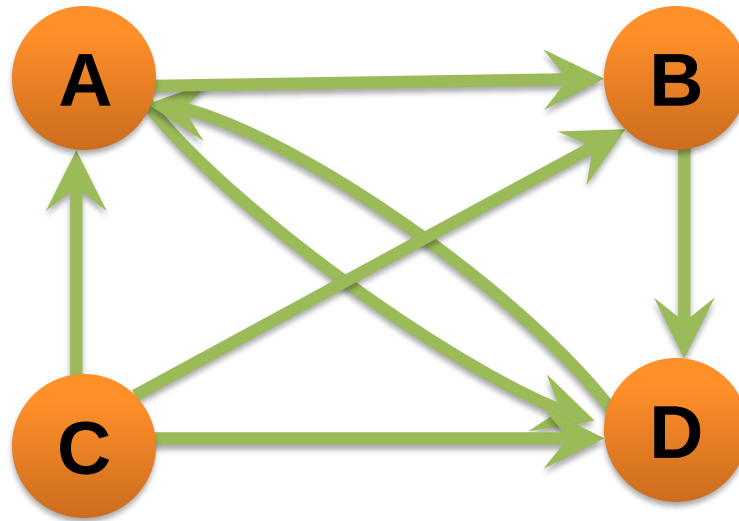
1. Adjacency Matrix



	A	B	C	D
A	0	1	0	1
B	0	0	1	1
C	0	0	0	0
D	0	0	1	1

For a digraph (= directed graph), the row sum is the out_degree, while the column sum is the in_degree

2. Adjacency List



Adjacency List

An undirected graph with n vertices and e edges has n head nodes and $2e$ list nodes in adjacency list

Nodes that have direct edges are only added as list nodes to that node.

In the above example, node A has direct edges with node B and node D

Graph Traversals

Graph traversal is a technique used for searching vertex in a graph. The graph traversal is also used to decide the order of vertices to be visited in the search process. A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of graph without visiting the vertices that are already visited.

Two types of graph traversals

1. Breadth first search (BFS)
2. Depth first search (DFS)

BFS and DFS produces a spanning tree as final result.

BFS are implemented using Queues and DFS using Stacks

Breadth-First Search Algorithm

Step 1: Start

Step 2: Read the graph

Step 3: Define a Queue of size = total number of vertices in the graph.

Step 4: Mark all the vertices as unvisited.

Step 5: Select any node as starting vertex (say **v_1**). Insert **v_1** into queue.

Step 6: Visit all the non-visited **adjacent** vertices of the vertex which is at front of the Queue and insert them into the Queue and mark them visited.

Step 7- When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.

Step 8 - Repeat steps 6 and 7 until queue becomes empty.

Step 9- When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

Step 10: stop

BFS Program

```
#include<stdio.h>
```

```
int a[20][20],q[20],visited[20],n,f=-1,r=-1;
```

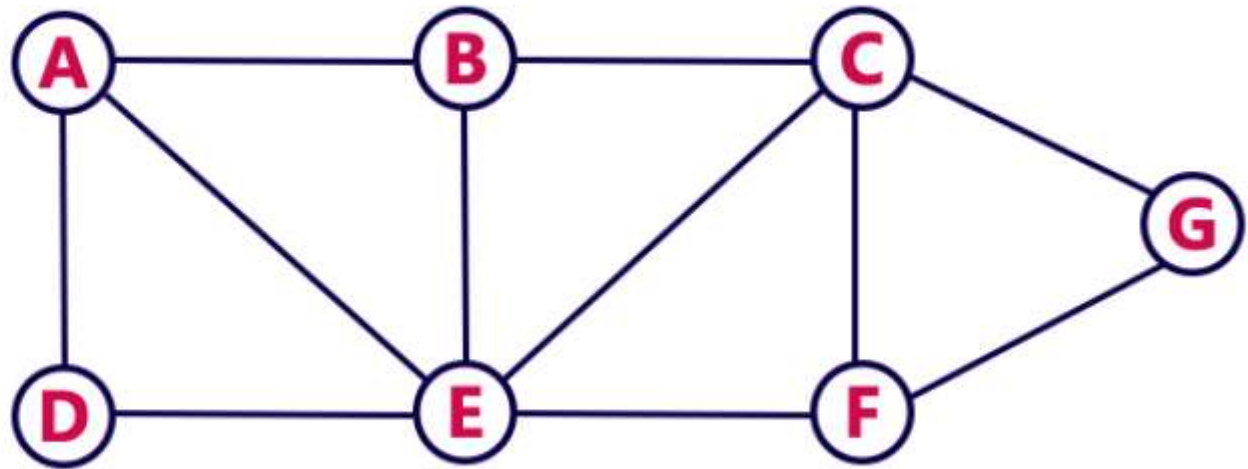
```
void bfs(int v)
```

```
{
    int i;
    for (i=0;i<n;i++) // check all the vertices in the graph
    {
        if(a[v][i] != 0 && visited[i] == 0) // adjacent to v and not visited
        {
            r=r+1;
            q[r]=i;           // insert them into queue
            visited[i]=1;     // mark the vertex visited
            printf("%d ",i);
        }
    }
    f=f+1;           // remove the vertex at front of the queue
    if(f<=r)         // as long as there are elements in the queue
        bfs(q[f]);   // perform bfs again on the vertex at front of the queue
}
```

Entire program is at below link

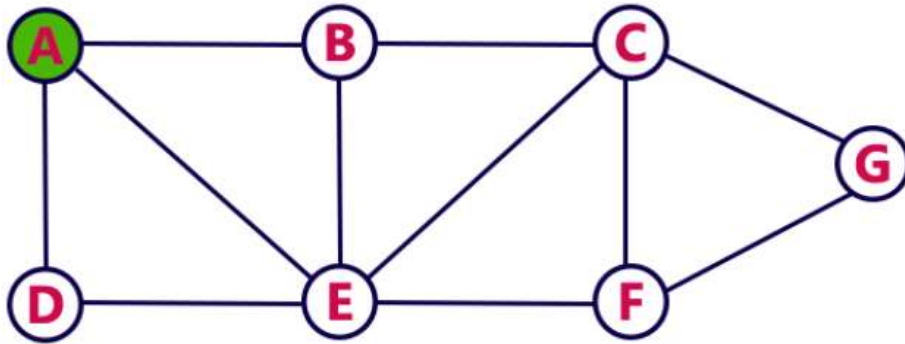
<http://enthusiaststudent.blogspot.com/2019/03/breadth-first-search-c-program.html>

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.



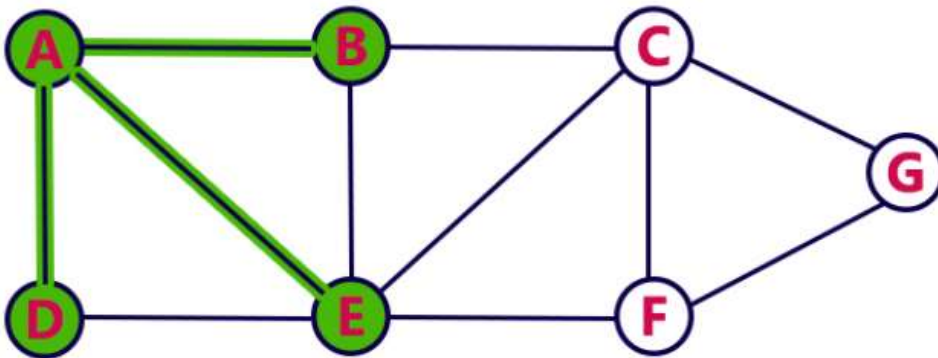
Queue



BFS traversal is A ->

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D, E, B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..



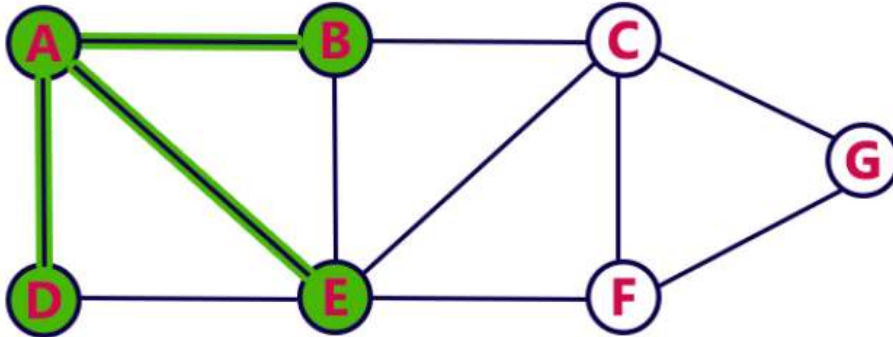
Queue



BFS traversal is A -> D -> E -> B

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.



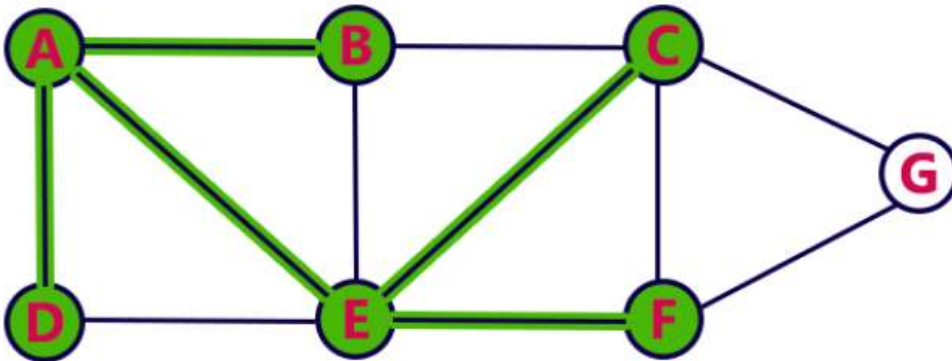
Queue



BFS traversal is A -> D -> E -> B

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.



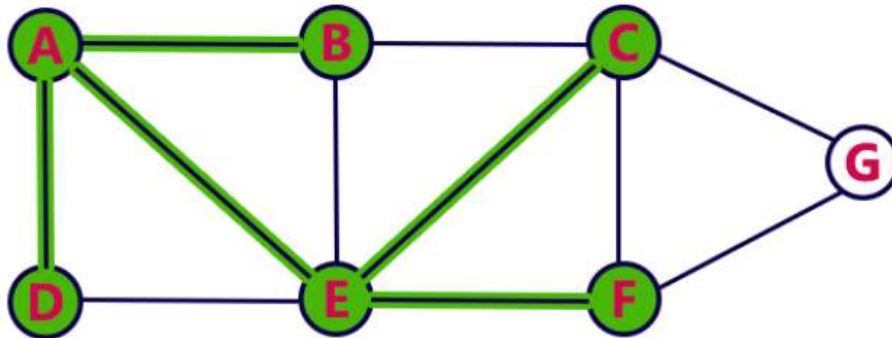
Queue



BFS traversal is A -> D -> E -> B -> C -> F

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.



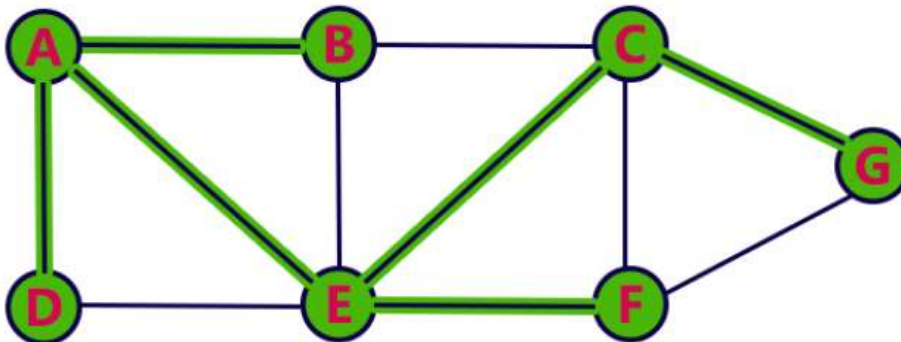
Queue



BFS traversal is A → D → E → B → C → F

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



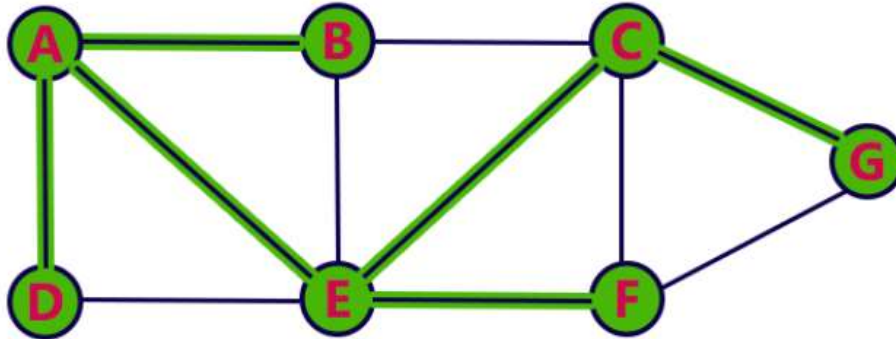
Queue



BFS traversal is A → D → E → B → C → F → G

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.



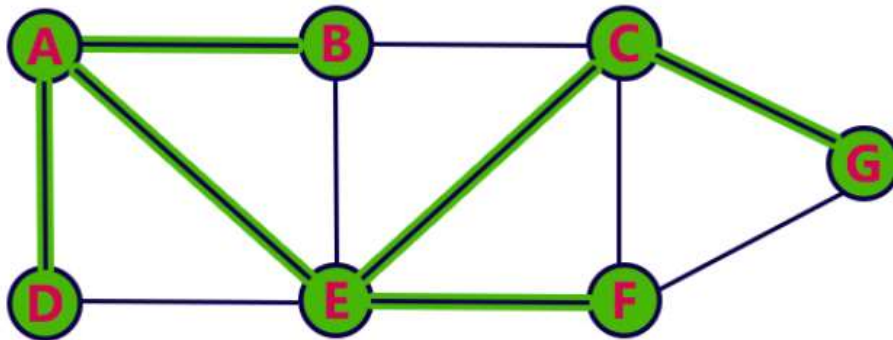
Queue



BFS traversal is A -> D -> E -> B -> C -> F -> G

Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.

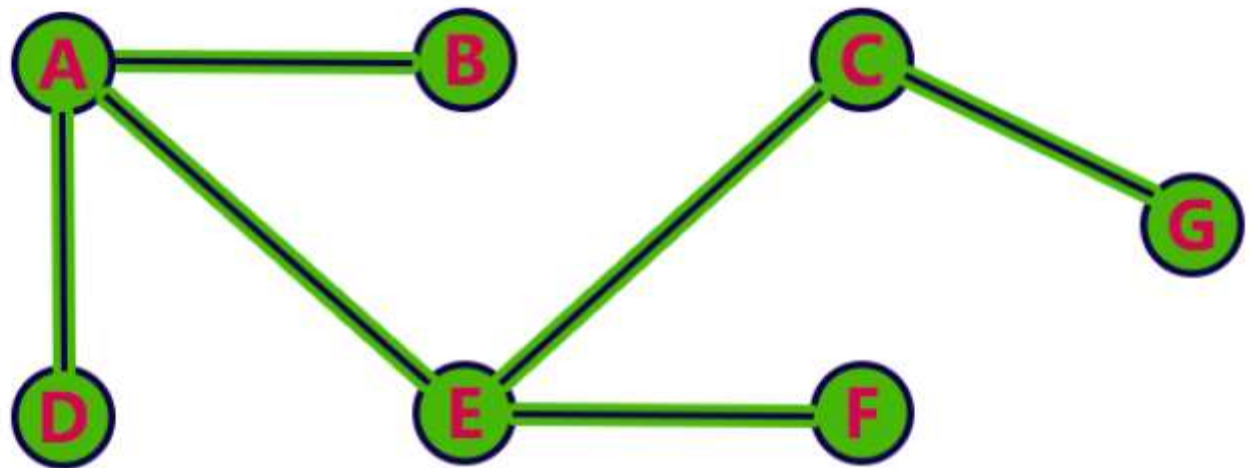


Queue



BFS traversal is A -> D -> E -> B -> C -> F -> G

- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



BFS traversal is A -> D -> E -> B -> C -> F -> G

Depth-First Search Algorithm

Step 1 - Define a Stack of size = total number of vertices in the graph.

Step 2 - Select any vertex as starting point for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack.

Step 4 - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

Step 5 - When there is no new vertex to visit then use back tracking and pop one vertex from the stack.

Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

DFS Program

```
#include<stdio.h>
```

```
int a[20][20],q[20],visited[20],n;
```

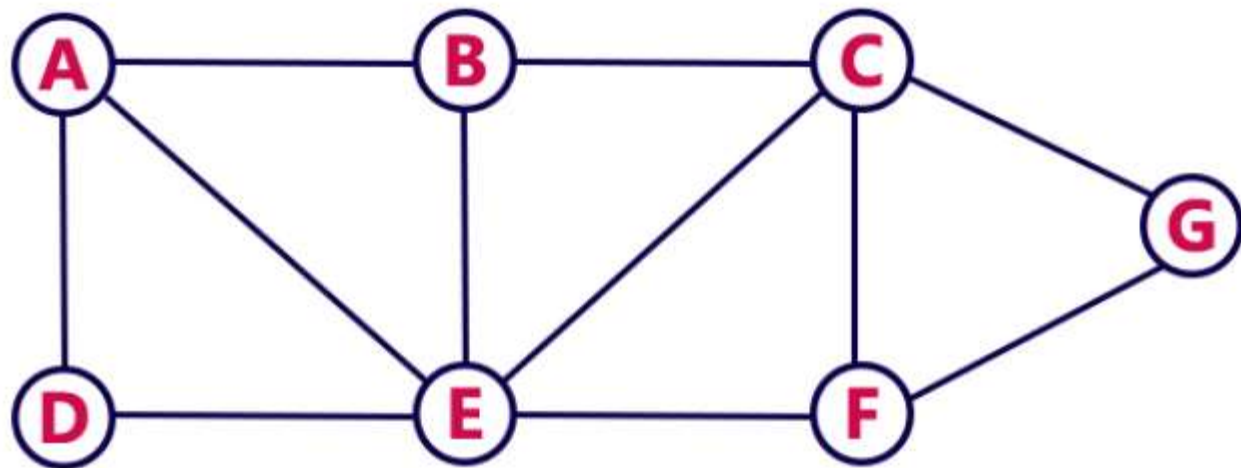
```
void dfs(int v)
```

```
{
    int i;
    for (i=0;i<n;i++)                // check all the vertices in the graph
    {
        if(a[v][i] != 0 && visited[i] == 0) // adjacent to v and not visited
        {
            visited[i]=1;                // mark the vertex visited
            printf("%d ",i);
            dfs(i);
        }
    }
}
```

Entire program is at below link

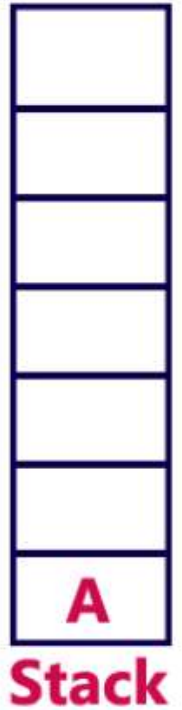
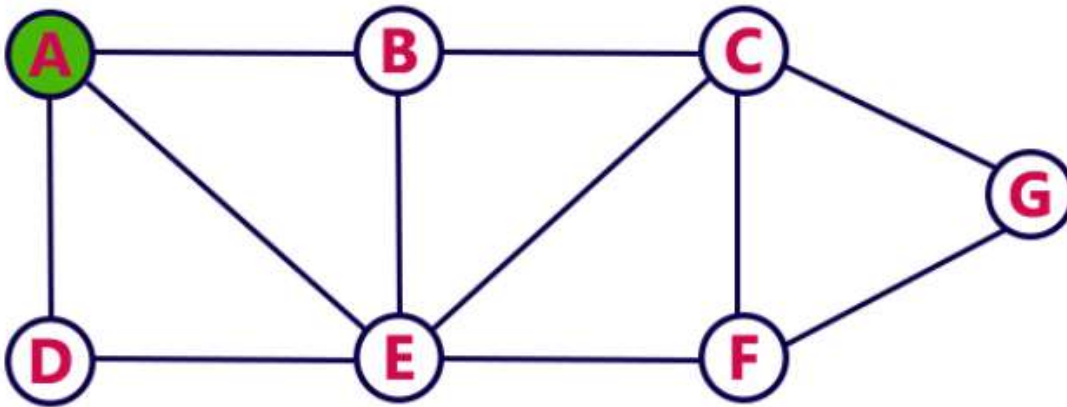
<http://enthusiaststudent.blogspot.com/2019/03/depth-first-search-c-program.html>

Consider the following example graph to perform DFS traversal



Step 1:

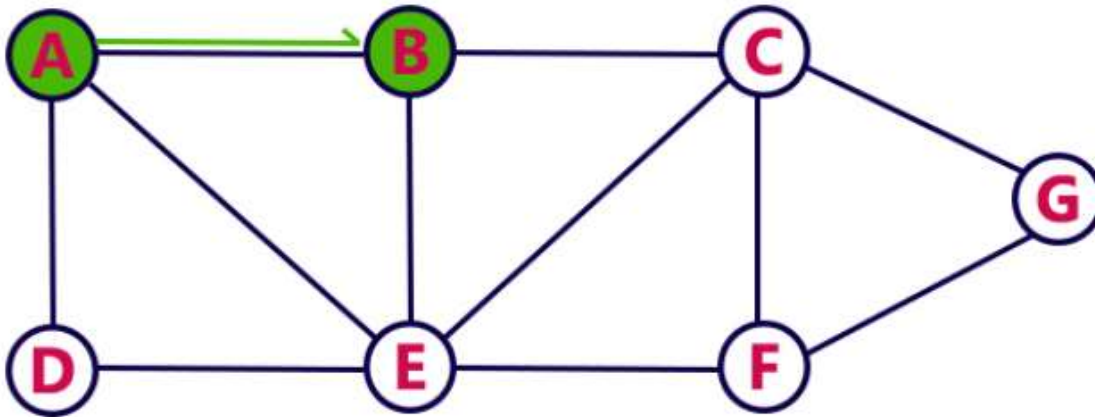
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



DFS traversal is A

Step 2:

- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.

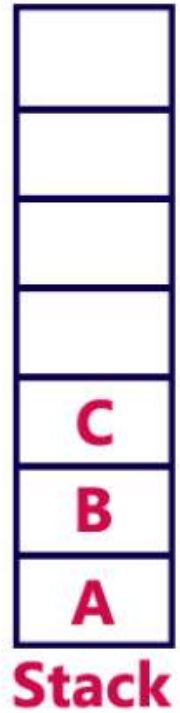
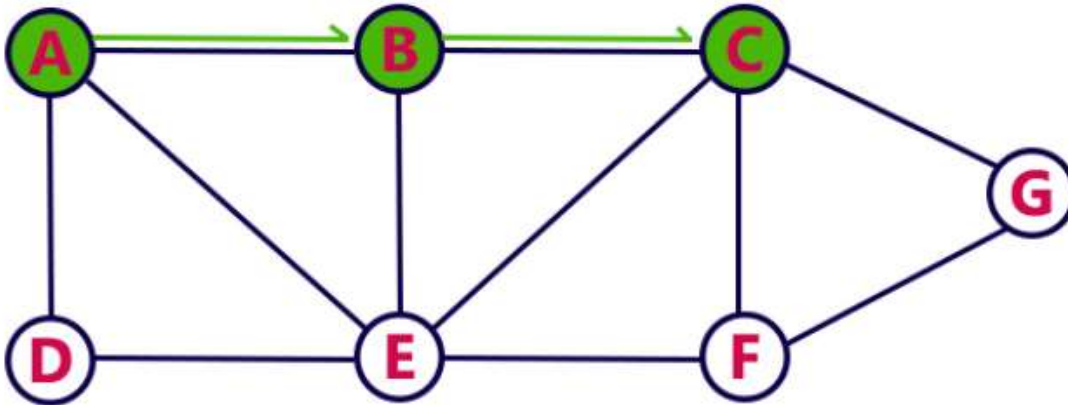


Stack

DFS traversal is A -> B

Step 3:

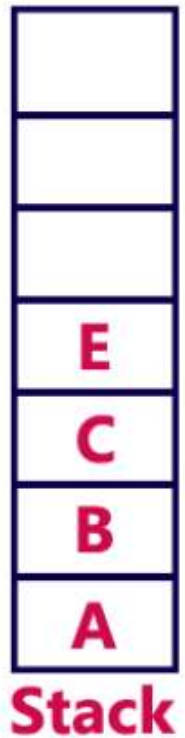
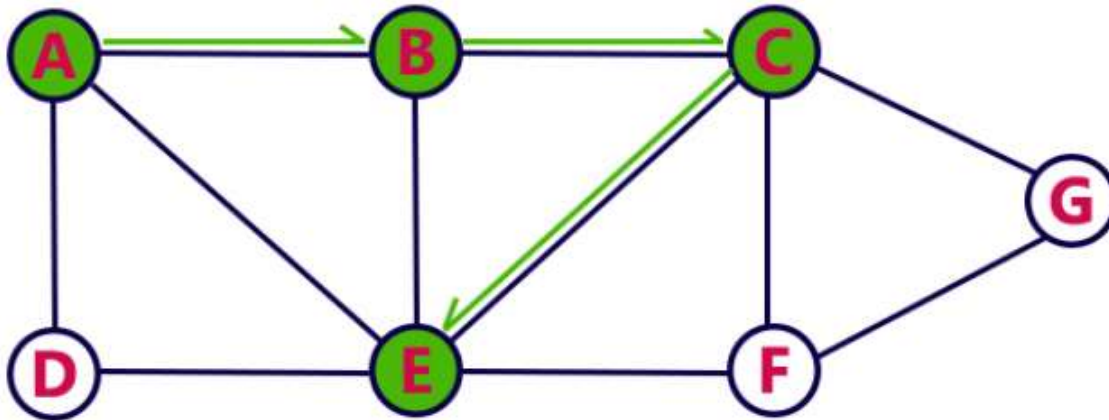
- Visit any adjacent vertex of **B** which is not visited (**C**).
- Push C on to the Stack.



DFS traversal is A -> B -> C

Step 4:

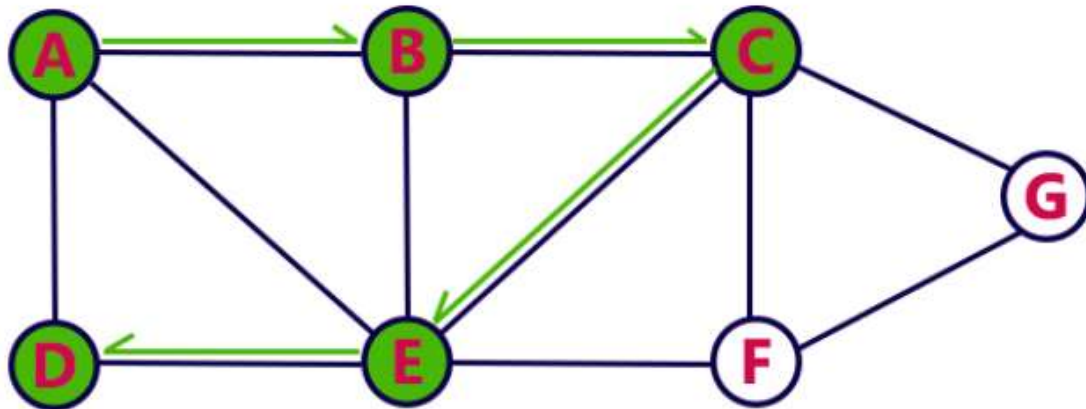
- Visit any adjacent vertex of **C** which is not visited (**E**).
- Push E on to the Stack



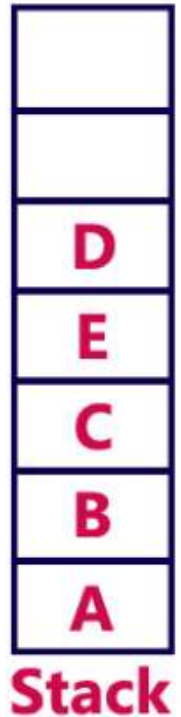
DFS traversal is A -> B -> C -> E

Step 5:

- Visit any adjacent vertex of **E** which is not visited (**D**).
- Push D on to the Stack

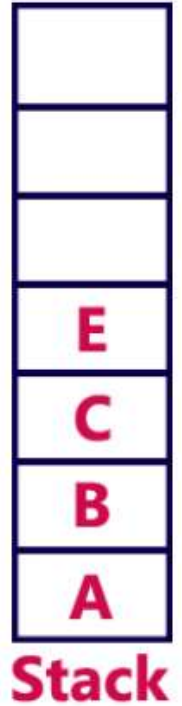
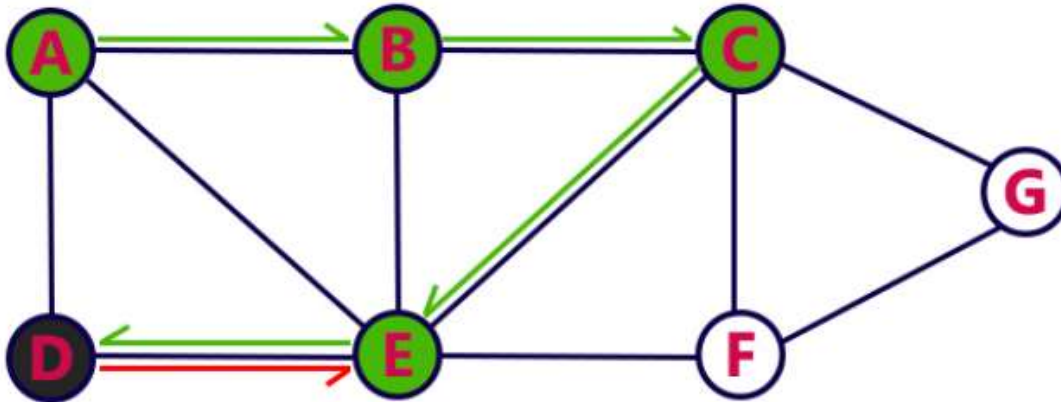


DFS traversal is A -> B -> C -> E -> D



Step 6:

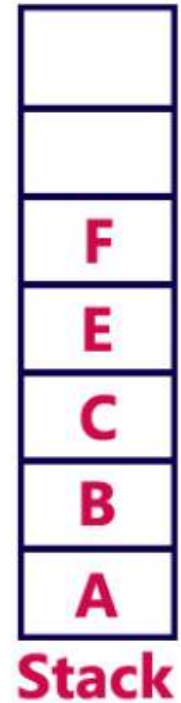
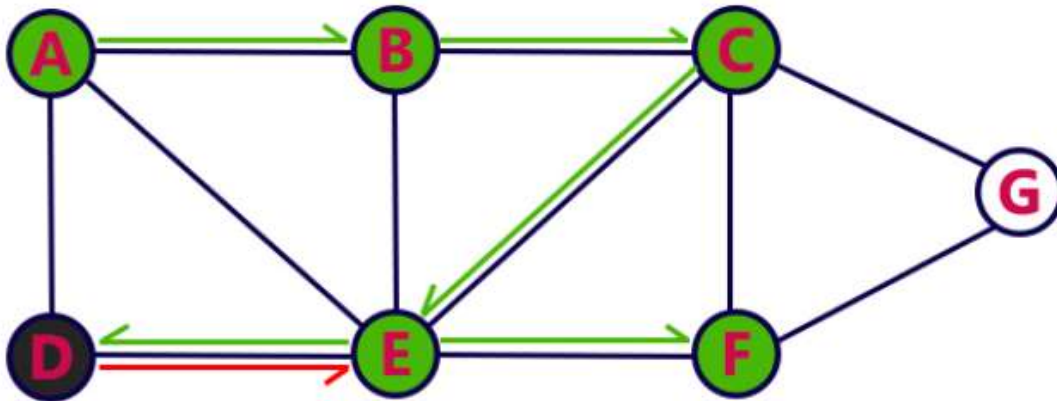
- There is no new vertex to be visited from D. So use back track.
- Pop D from the Stack.



DFS traversal is A -> B -> C -> E -> D

Step 7:

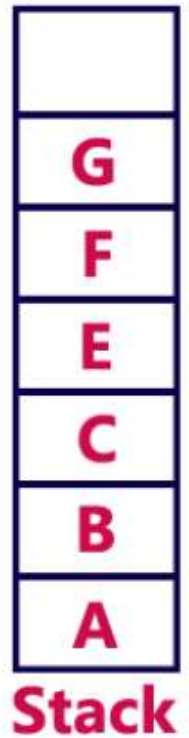
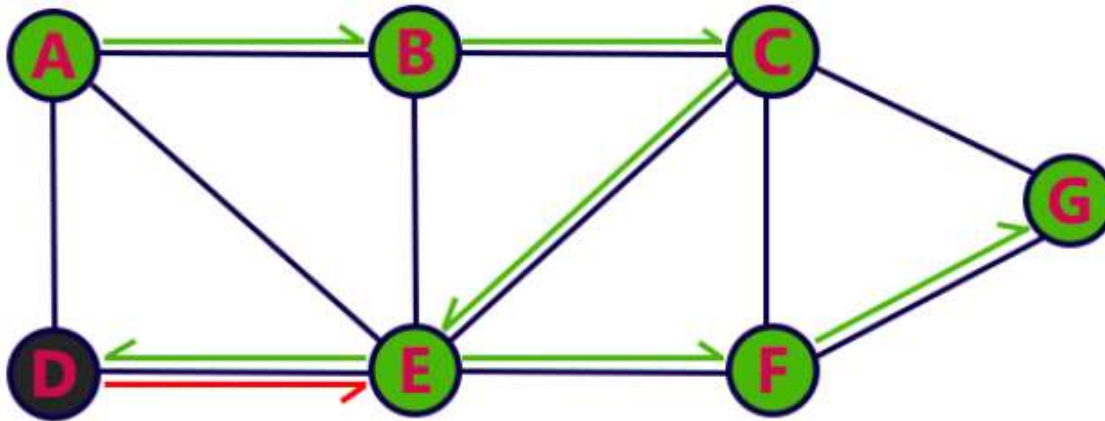
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.



DFS traversal is A -> B -> C -> E -> D -> F

Step 8:

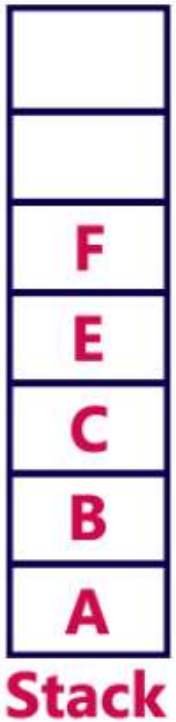
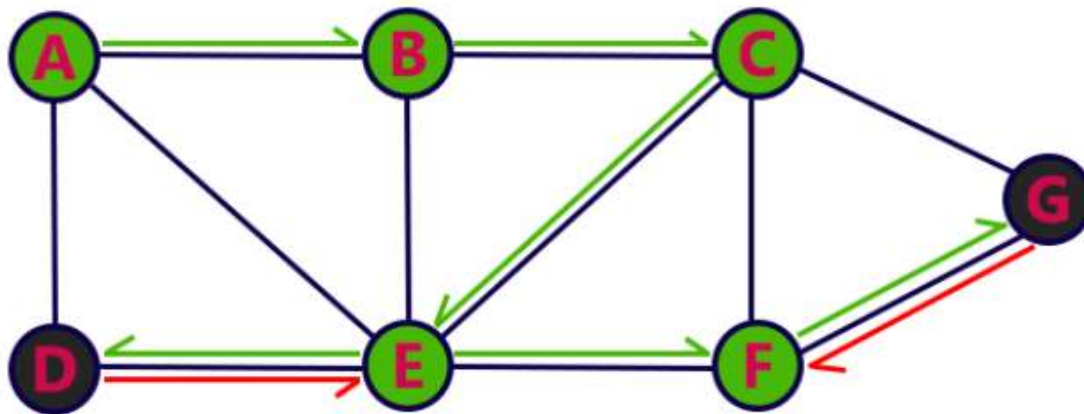
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 9:

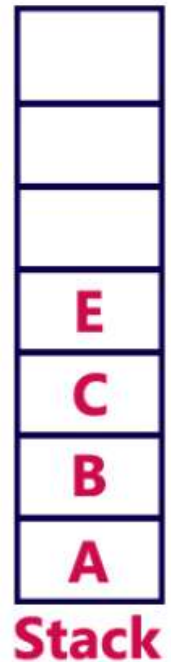
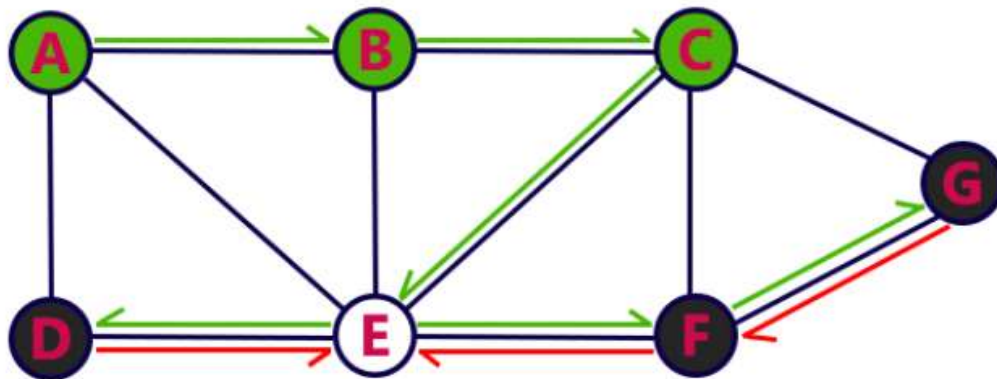
- There is no new vertex to be visited from G. So use back track.
- Pop G from the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 10:

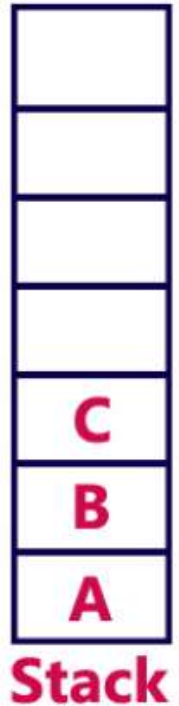
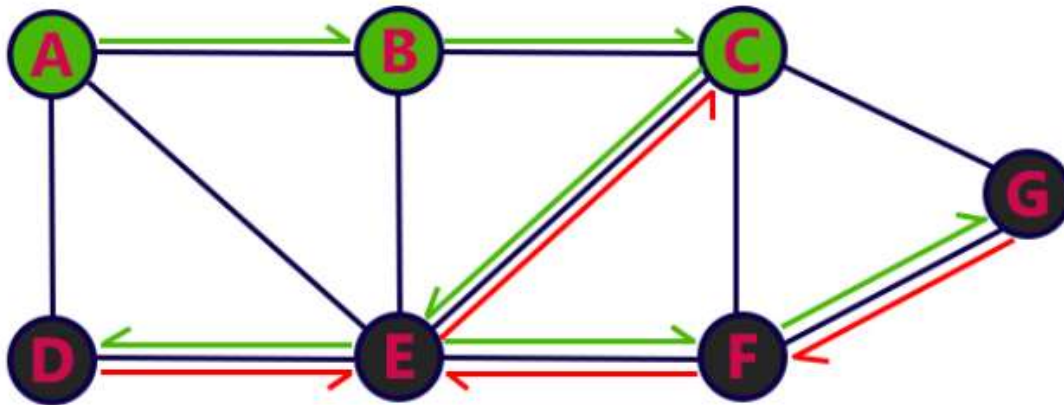
- There is no new vertex to be visited from F. So use back track.
- Pop F from the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 11:

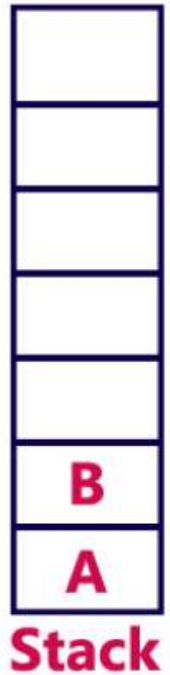
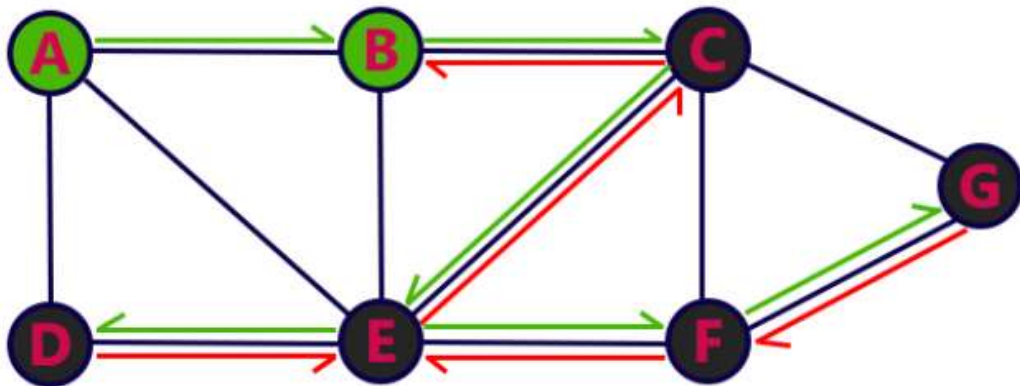
- There is no new vertex to be visited from E. So use back track.
- Pop E from the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 12:

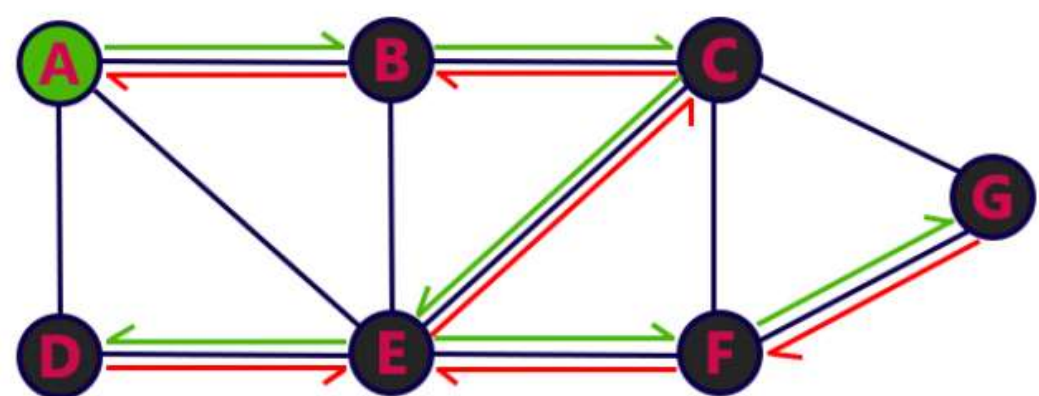
- There is no new vertex to be visited from C. So use back track.
- Pop C from the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 13:

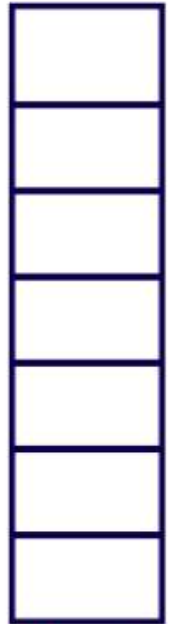
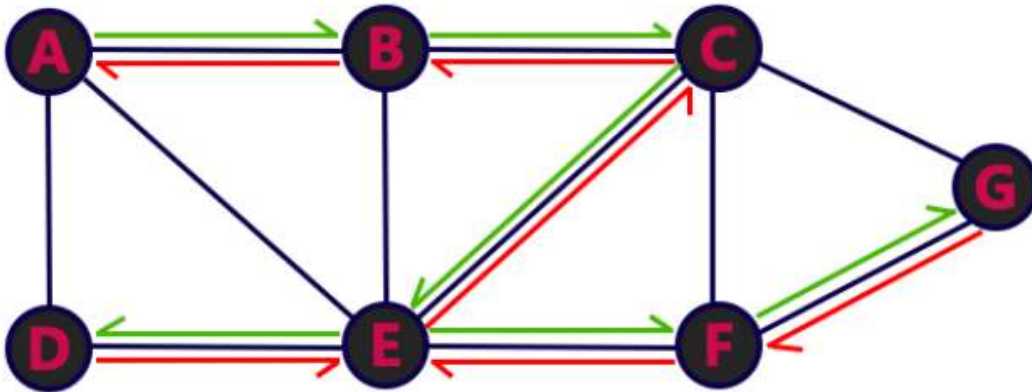
- There is no new vertex to be visited from B. So use back track.
- Pop B from the Stack.



DFS traversal is A -> B -> C -> E -> D -> F -> G

Step 14:

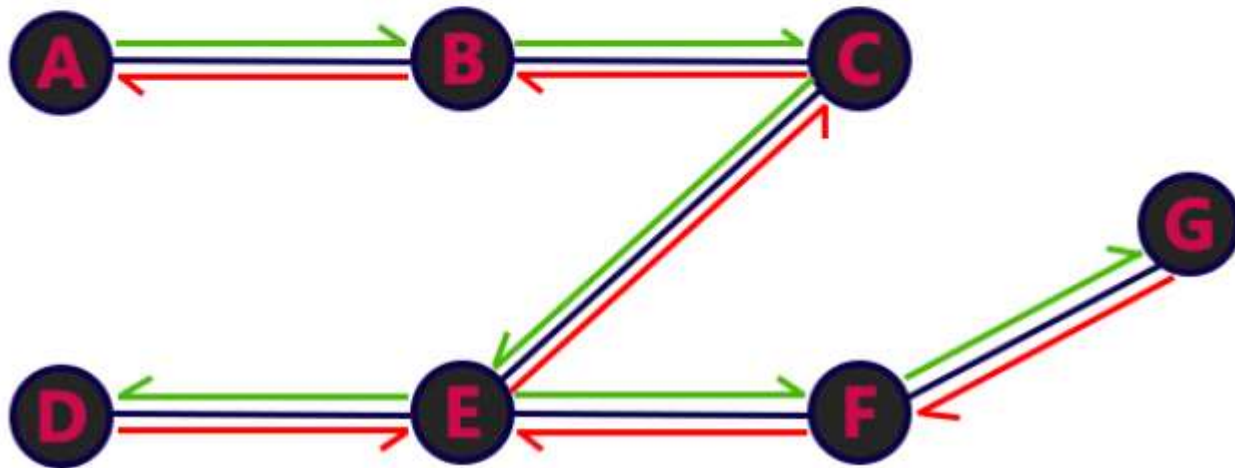
- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.



Stack

DFS traversal is A -> B -> C -> E -> D -> F -> G

- Stack became Empty. So stop DFS Traversal.
- Final result of DFS traversal is following spanning tree.

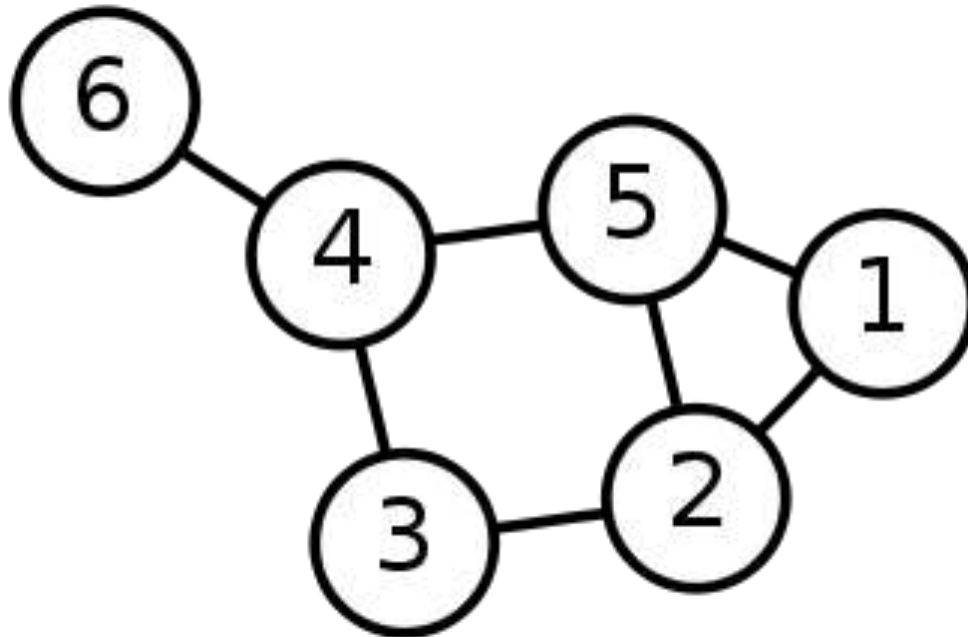


DFS traversal is A -> B -> C -> E -> D -> F -> G

Dijkstra's algorithm

Single-Source Shortest Path Problem

Single-Source Shortest Path Problem - The problem of finding shortest paths from a source vertex v to all other vertices in the graph.



Dijkstra's algorithm

Dijkstra's algorithm - is a solution to the single-source shortest path problem in graph theory.

Works on both directed and undirected graphs. However, all edges must have nonnegative weights.

Approach: Greedy

Input: Weighted graph $G=\{E,V\}$ and source vertex $v \in V$, such that all edge weights are nonnegative

Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

Dijkstra's algorithm - Pseudocode

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.\text{Adj}[u]$ 
8          RELAX( $u, v, w$ )
```

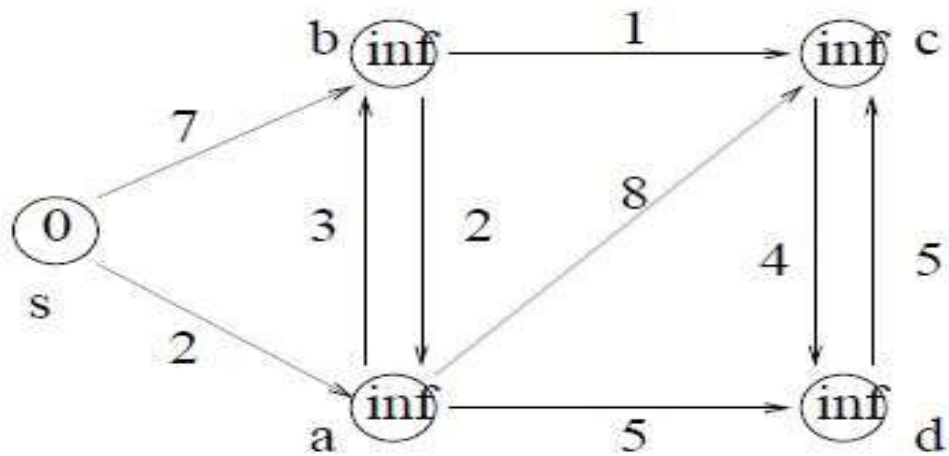
INITIALIZE-SINGLE-SOURCE(G, s)

```
1  for each vertex  $v \in G.V$ 
2       $v.d = \infty$ 
3       $v.\pi = \text{NIL}$ 
4   $s.d = 0$ 
```

RELAX(u, v, w)

```
1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
```

Relaxation. If the new path from s to v is shorter than $d[v]$, then update $d[v]$ to the length of this new path.

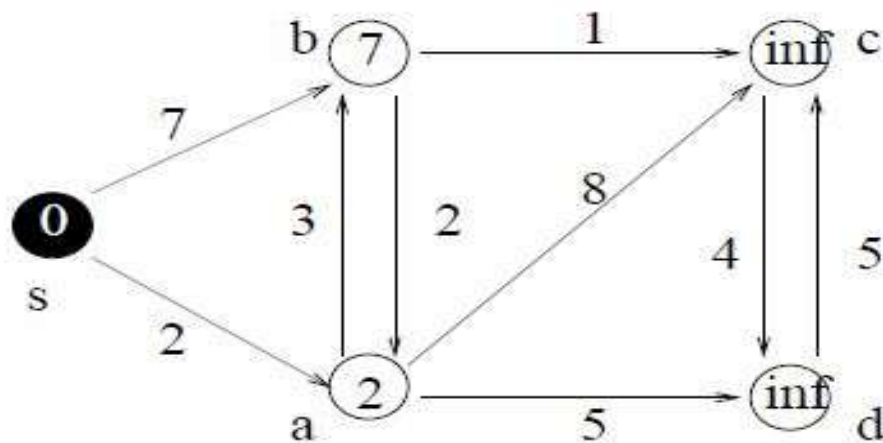


Step 0: Initialization.

v	s	a	b	c	d
$d[v]$	0	∞	∞	∞	∞
$pred[v]$	nil	nil	nil	nil	nil
$color[v]$	W	W	W	W	W

Priority Queue:

v	s	a	b	c	d
$d[v]$	0	∞	∞	∞	∞

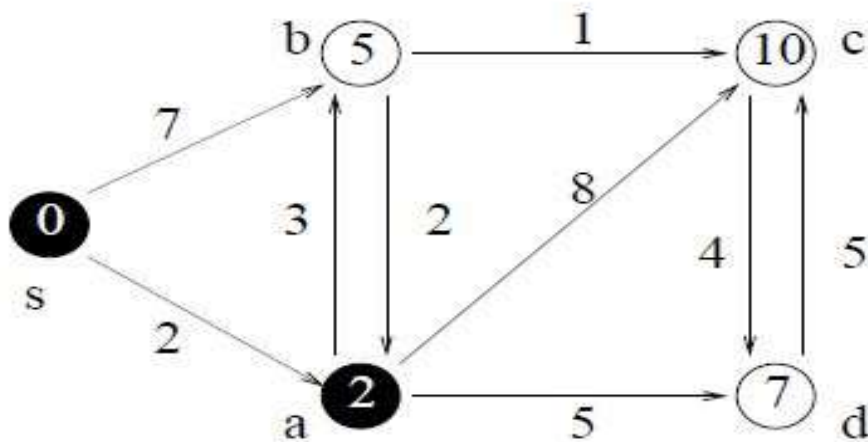


Step 1: As $Adj[s] = \{a, b\}$, work on a and b and update information.

v	s	a	b	c	d
$d[v]$	0	2	7	∞	∞
$pred[v]$	nil	s	s	nil	nil
$color[v]$	B	W	W	W	W

Priority Queue:

v	a	b	c	d
$d[v]$	2	7	∞	∞

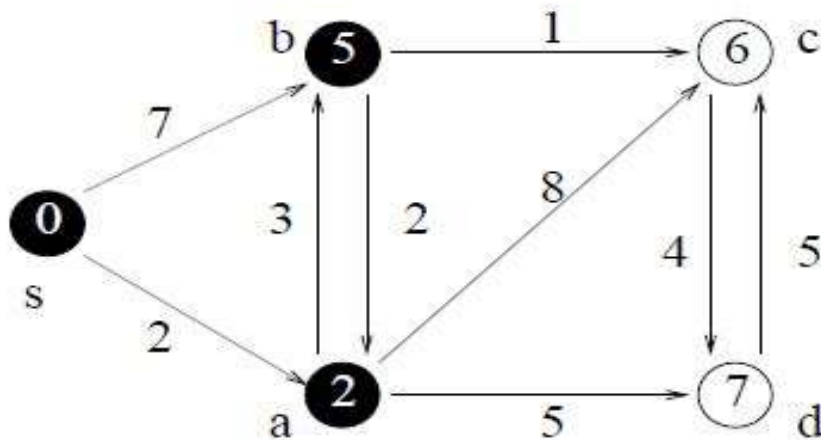


Step 2: After Step 1, a has the minimum key in the priority queue. As $Adj[a] = \{b, c, d\}$, work on b, c, d and update information.

v	s	a	b	c	d
$d[v]$	0	2	5	10	7
$pred[v]$	nil	s	a	a	a
$color[v]$	B	B	W	W	W

Priority Queue:

v	b	c	d
$d[v]$	5	10	7

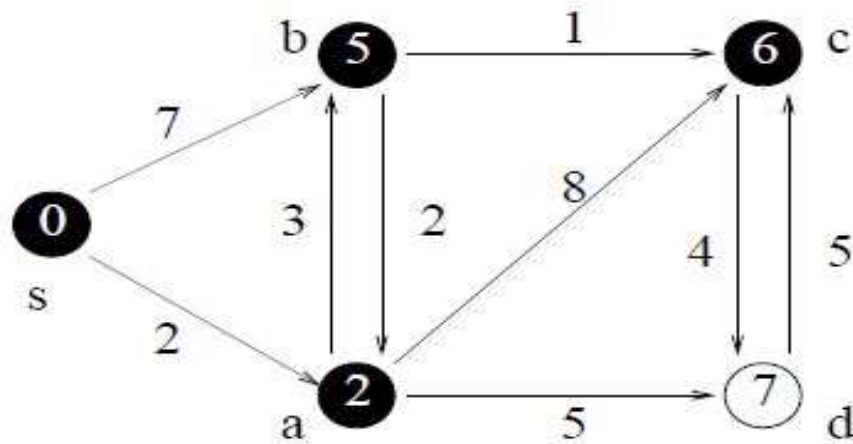


Step 3: After Step 2, b has the minimum key in the priority queue. As $Adj[b] = \{a, c\}$, work on a, c and update information.

v	s	a	b	c	d
$d[v]$	0	2	5	6	7
$pred[v]$	nil	s	a	b	a
$color[v]$	B	B	B	W	W

Priority Queue:

v	c	d
$d[v]$	6	7

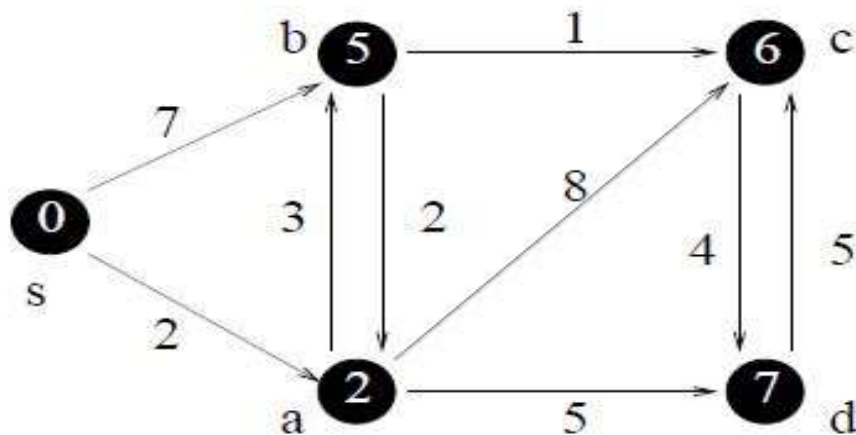


Step 4: After Step 3, c has the minimum key in the priority queue. As $Adj[c] = \{d\}$, work on d and update information.

v	s	a	b	c	d
$d[v]$	0	2	5	6	7
$pred[v]$	nil	s	a	b	a
$color[v]$	B	B	B	B	W

Priority Queue:

v	d
$d[v]$	7

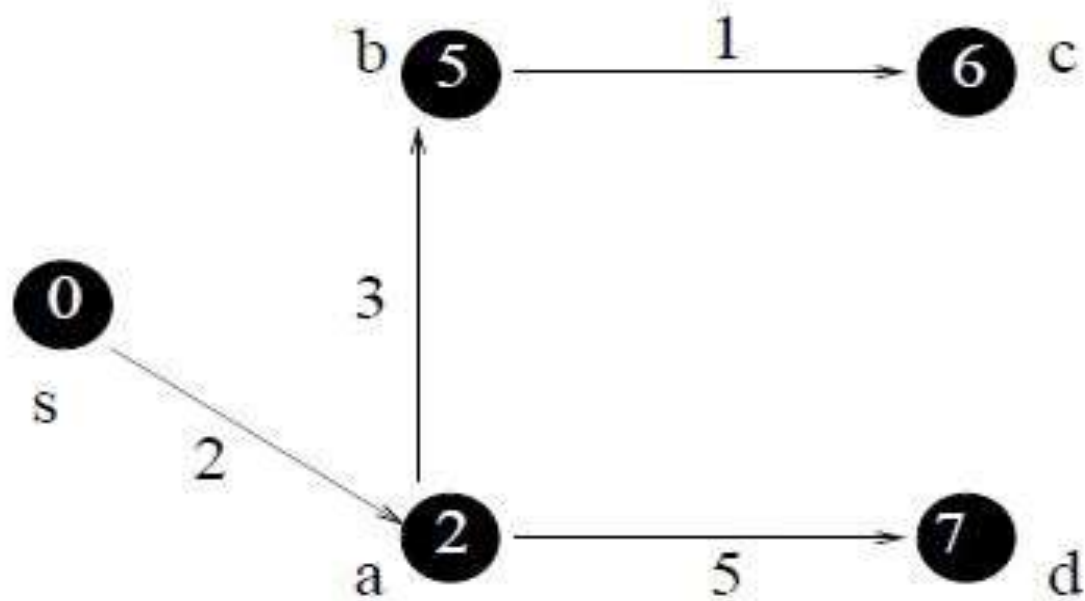


Step 5: After Step 4, d has the minimum key in the priority queue. As $Adj[d] = \{c\}$, work on c and update information.

v	s	a	b	c	d
$d[v]$	0	2	5	6	7
$pred[v]$	nil	s	a	b	a
$color[v]$	B	B	B	B	B

Priority Queue: $Q = \emptyset$.

We are done.



Example:

v	s	a	b	c	d
$d[v]$	0	2	5	6	7
$pred[v]$	nil	s	a	b	a

Refer the link

https://youtu.be/HlFsexTpM4Q?si=O4_jlRrDN70HXL4d

https://youtu.be/JcN_nq1EAr4?si=x89RM8W6KzF36-IG